

Python и Data Science?

EAI 2026



Курс 2026 : Правила игры

Лабораторные работы	Рубежка	Экзамен
6 работ 12 баллов за работу 72 балла суммарно	8 баллов Будет на какой-то из лекций :)	20 баллов 10 за теорию + 10 за практику

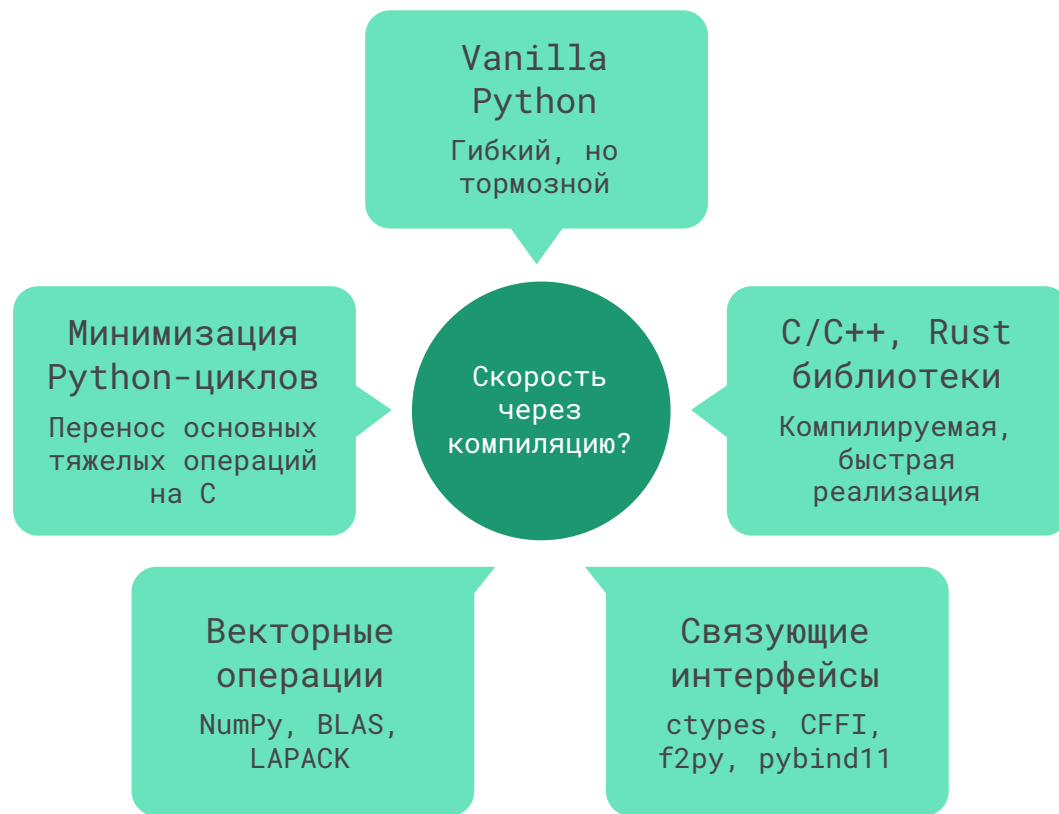
Курс 2026 : Правила игры

Парадокс производительности Python

Python – **динамически
типизированный, интерпретируемый**
язык программирования

Другими словами, он – **медленный**.

*(Ну или по крайней мере более
медленный, чем другие языки,
которые можно было бы использовать
для обработки больших объемов
данных)*



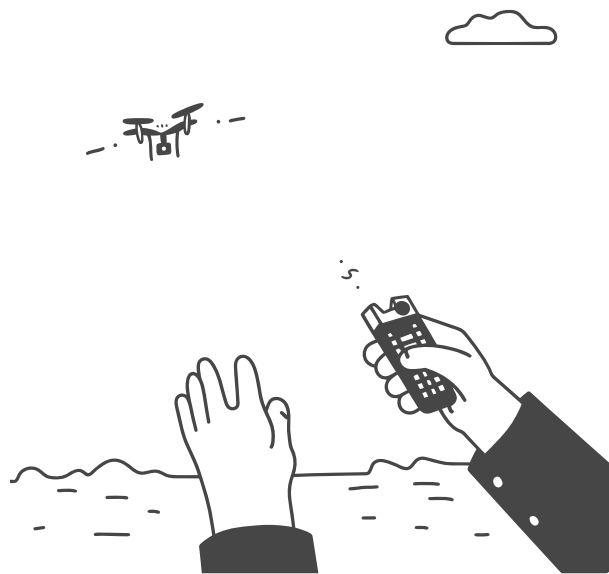


Проблема двух языков

Data Science на то и Science:

- **Нужно делать много экспериментов** → Скриптовые интерпретируемые языки, можно быстро делать много итераций (MATLAB, R)
- Когда эксперименты уже окончены, их результатами нужно пользоваться на очень больших наборах данных, **производительность становится критичнее** → Реализация финальных алгоритмов на компилируемых языках (C++, Fortran)

Решение от Гвидо ван Россума



Python изначально проектировался с прицелом на глубокую интеграцию с кодом, написанном на C

(начиная с недавних пор еще и с Rust, но это совсем другая история)

Скрипт на Python как «пульт управления»:

- Организация потоков данных
- Управление логикой
- Вызов заранее оптимизированных функций

Python как «связующий язык»

Высокоуровневые абстракции

Фокус на логике обработки данных, а не на эффективной реализации

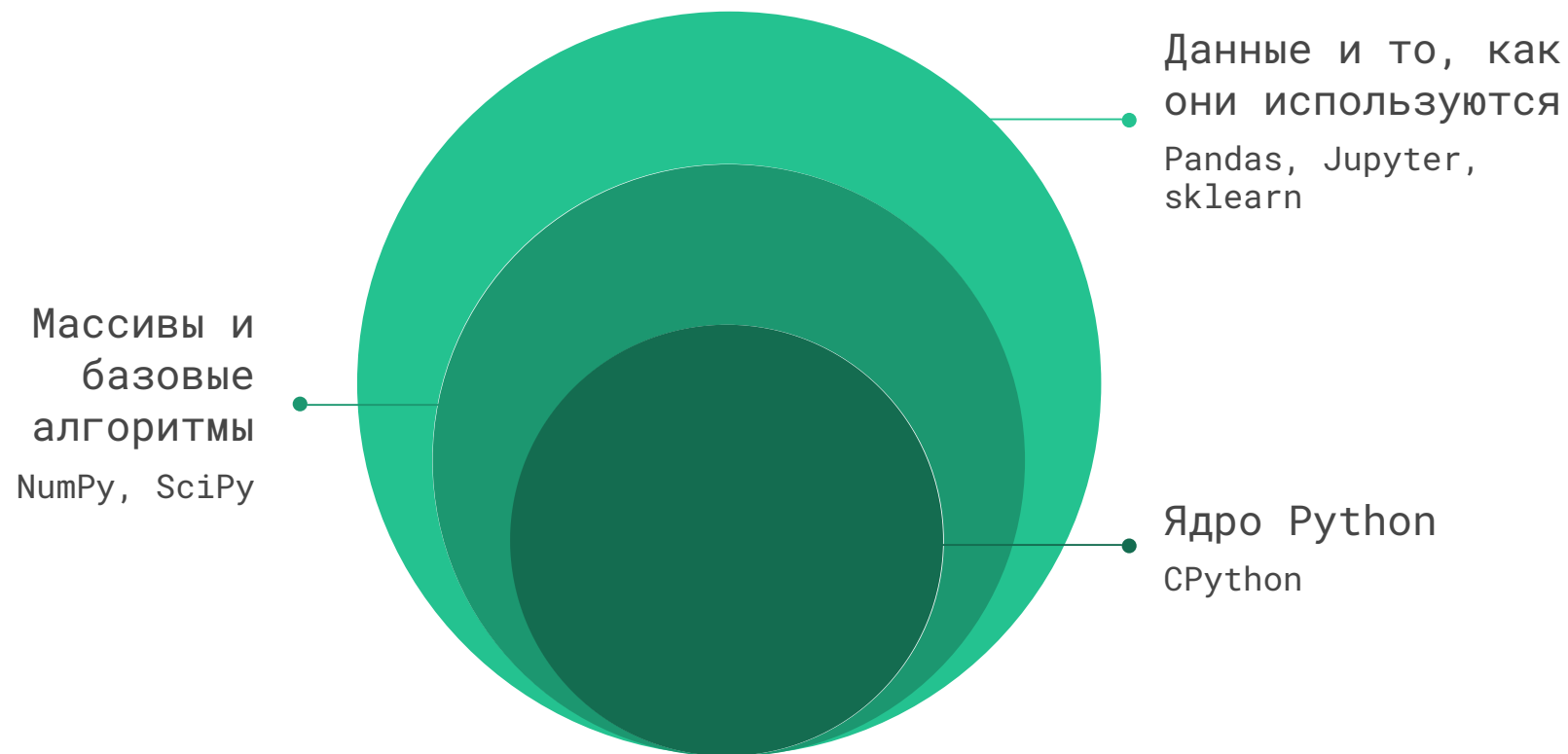
Интеграция с C/C++

Легкое взаимодействие с низкоуровневым кодом

Оптимизированные библиотеки

Производительность на уровне компилируемых языков

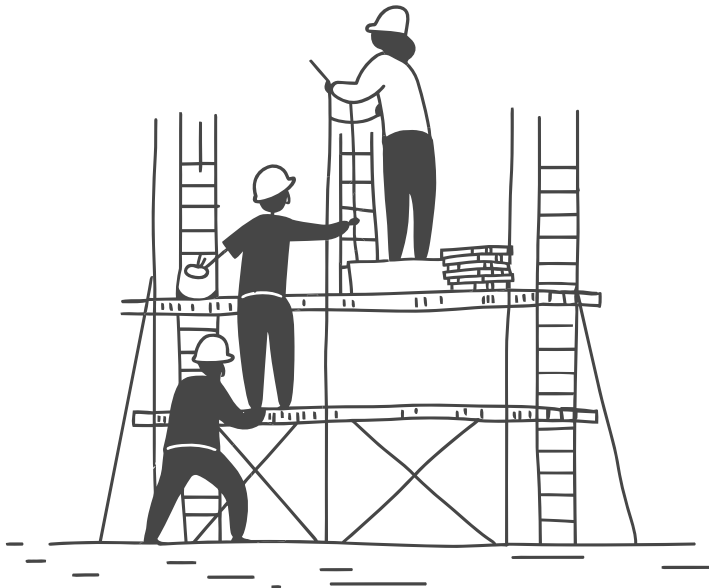
SciPy Stack



Уровень 1: Python (CPython)

Собственно, сам интерпретатор

- Синтаксис языка и семантика
- Управление памятью объектов
- API для взаимодействия с кодом на C
- Базовая инфраструктура для всех библиотек

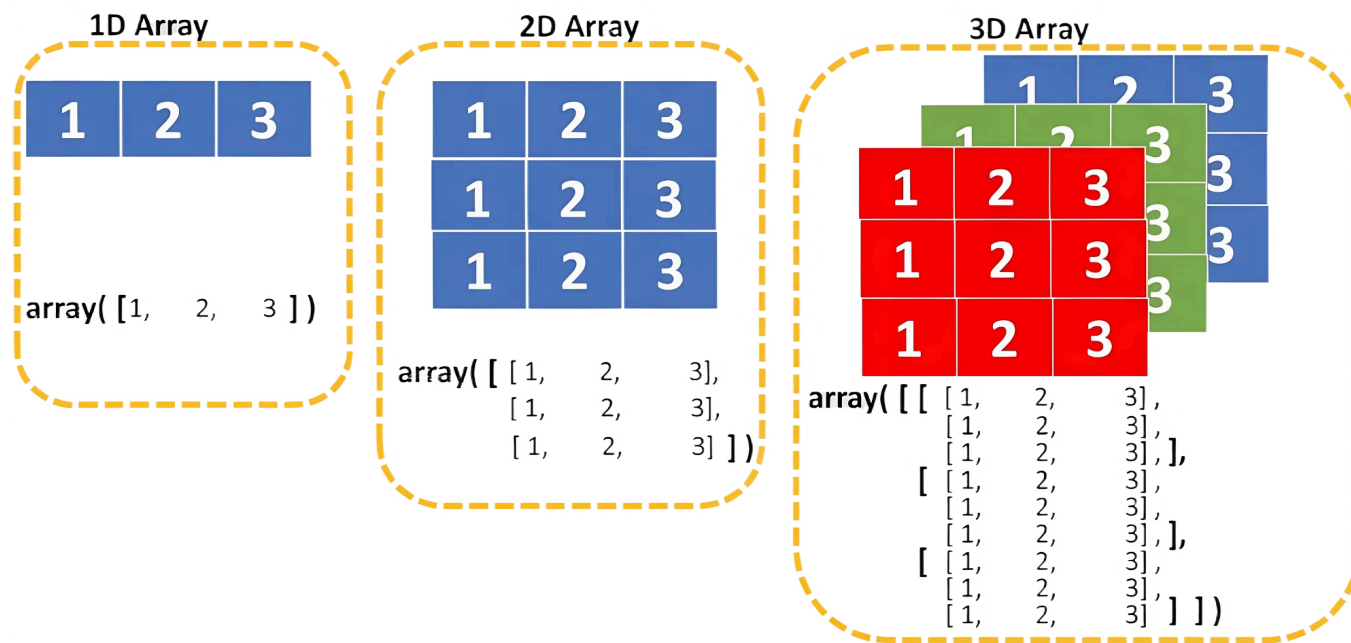


Уровень 2: NumPy (Numerical Python)

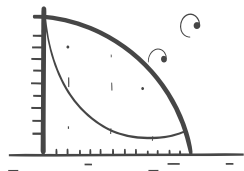
Объект многомерного массива (`ndarray`)

- Непрерывное хранение в памяти
- Векторизованные операции
- Решение проблемы производительности списков Python

Ускорение в 50-100 раз по сравнению со списками



Уровень 3: SciPy (Scientific Python)



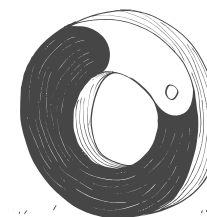
Интегрирование

Выполнение численного интегрирования для вычисления площадей под кривыми и решения дифференциальных уравнений.



Оптимизация

Поиск минимумов или максимумов функций и подгонка моделей к данным с использованием различных алгоритмов.



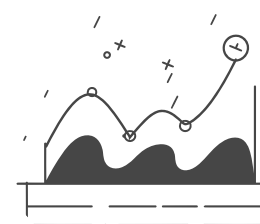
Линейная алгебра

Эффективное выполнение операций с матрицами, решение систем линейных уравнений и вычисление собственных значений.



Обработка сигналов

Применение фильтров, преобразований (например, БПФ) и спектрального анализа к сигналам.



Интерполяция

Оценка значений между известными точками данных с использованием различных методов, таких как сплайны.

Уровень 4:

Pandas

Работа с табличными данными

Ключевые абстракции:

- `DataFrame` – таблица с метками
- `Series` – одномерный массив
- Индексы для выравнивания данных
- Поддержка разнородных типов



Уровень 5: Высокоуровневый инструментарий

Matplotlib

Визуализация данных и
построение графиков

Seaborn

Статистические визуализации

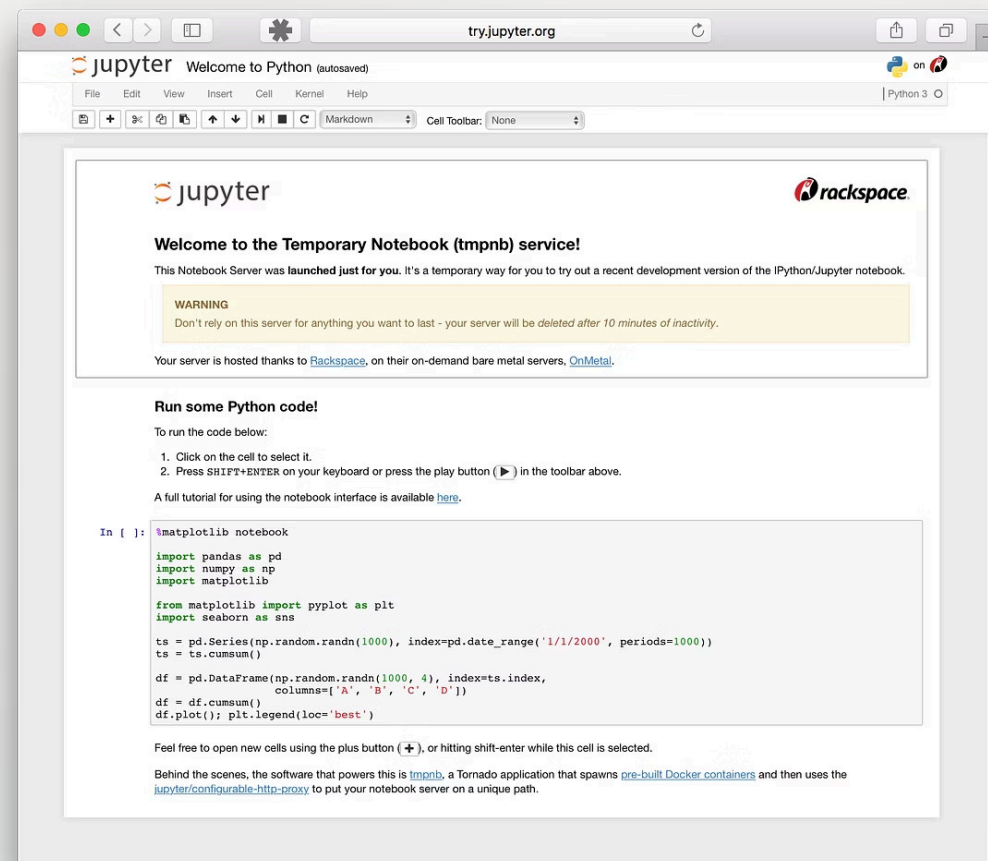
Scikit-Learn

Машинное обучение и
моделирование

Интерфейс : IPython и Jupyter

Интерактивная среда, объединяющая код,
текст и визуализацию

Обеспечивает воспроизводимость
экспериментов



ЧАСТЬ I

Интерактивные вычисления

IPython и Jupyter

Проблема цикла «edit-compile-run»

Анализ данных – итеративный процесс

- Загрузка данных (минуты)
- Исследование и визуализация
- Обнаружение аномалий
- Фильтрация и повторный анализ



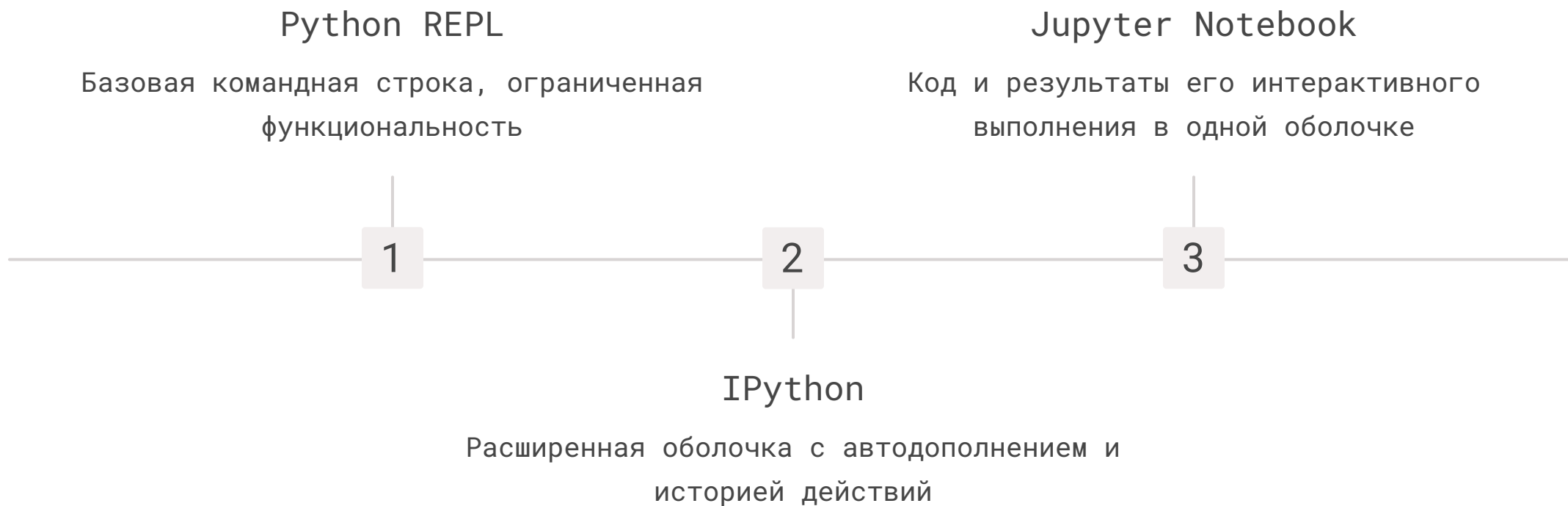
Неэффективно

Перезапуск всего скрипта на каждой итерации

✓ Решение

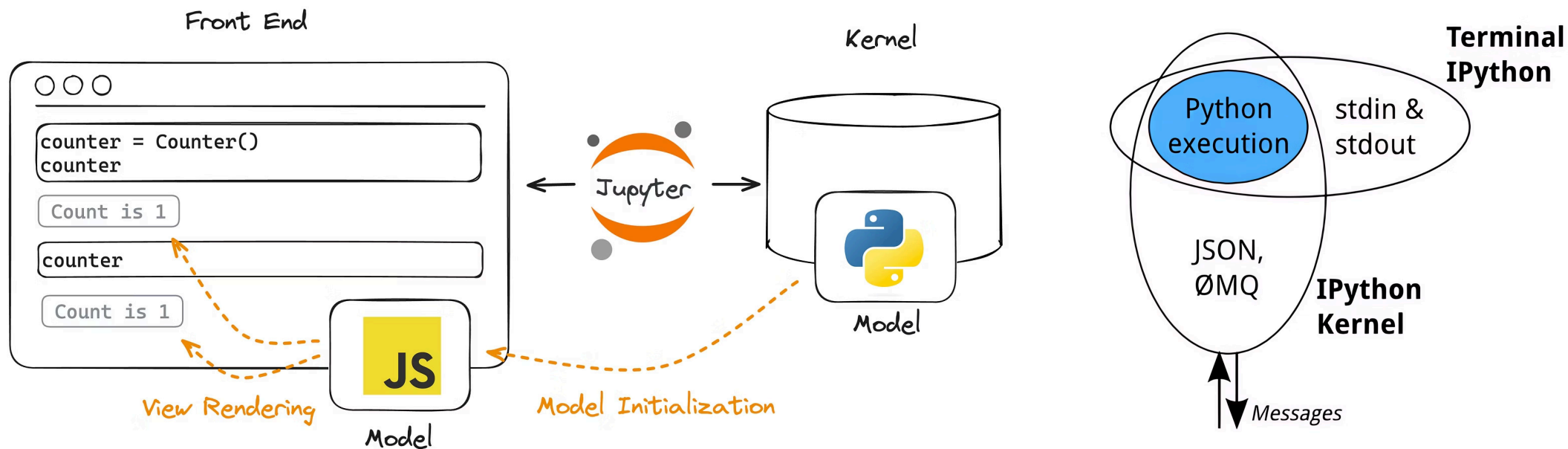
Интерактивная оболочка с сохранением состояния

Эволюция: от REPL к Notebook



Jupyter (Julia, Python, R)

Разделение интерфейса и ядра вычислений



Три главных компонента Jupyter



Фронтенд (Frontend)

Интерфейс в браузере



Ядро (Kernel)

Процесс, который исполняет код



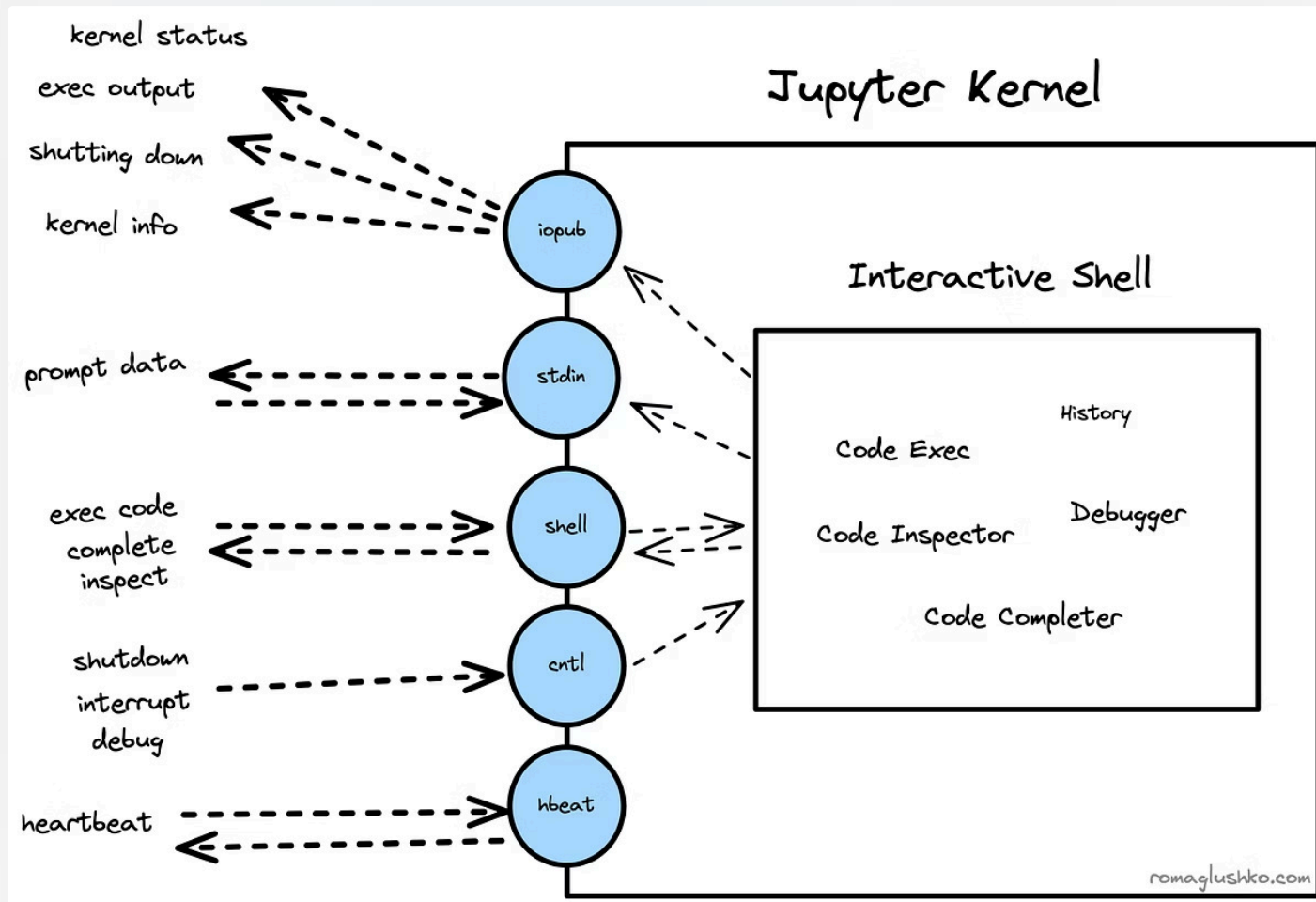
Сервер сообщений

ZeroMQ

ZeroMQ

Высокопроизводительная
асинхронная библиотека для
обмена сообщениями

Связь с вычислительным ядром
через 5 специализированных
каналов (сокетов)



Вот они, слева направо:

1

Shell

Основной канал исполнения кода (ROUTER/DEALER)

2

IOPub

Публикация результатов и вывода (PUB/SUB)

3

Stdin

Интерактивный ввод данных (ROUTER/DEALER)

4

Control

Команды управления и прерывания (ROUTER/DEALER)

5

Heartbeat

Мониторинг состояния ядра (REQ/REP)

Shell – основной канал исполнения

ROUTER/DEALER
(двунаправленный)

Функции:

- Прием запросов `execute_request`
- Последовательная обработка
- Блокировка при длительных вычислениях

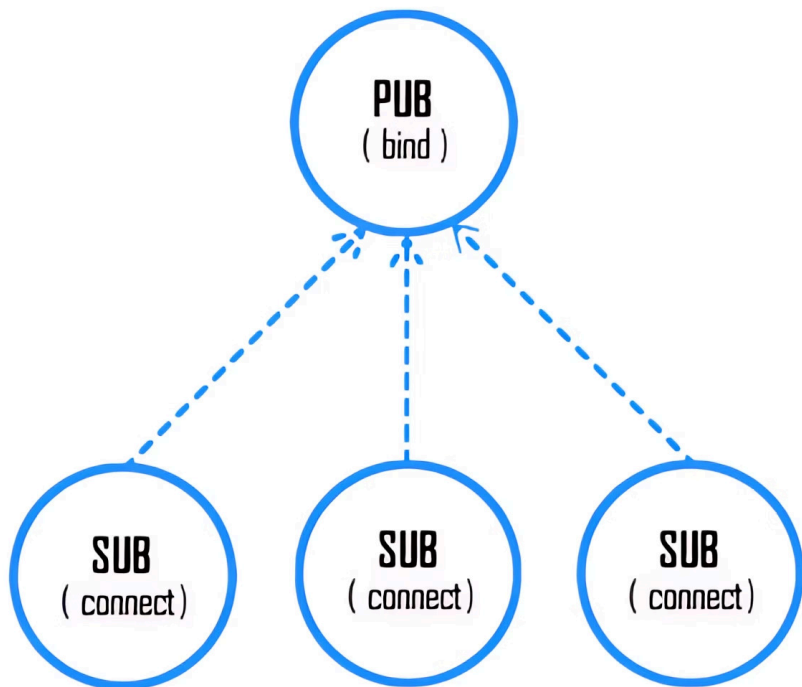
Почему ячейки ждут?

Пока ядро занято, Shell-сокеты заблокированы

Новые ячейки помечаются [*] и ждут освобождения

```
[7]  while True:  
    ...
```

IOPub – канал публикации



PUB/SUB (однонаправленный)

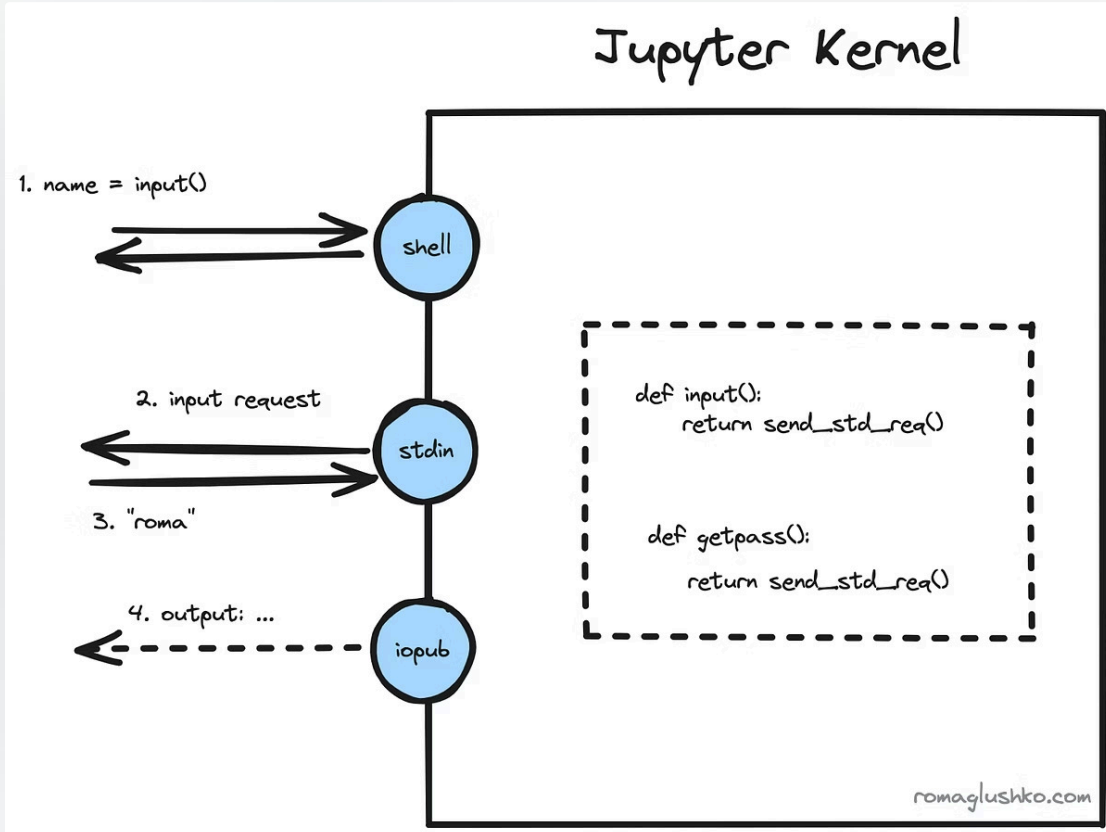
Вещательная модель

Содержимое:

- Вывод stdout/stderr (print)
- Графики и визуализации
- Сообщения об ошибках
- Обновления статуса выполнения

Несколько клиентов могут наблюдать за одним ядром

Stdin и Control



Stdin (ROUTER/DEALER)

Интерактивный ввод

- Функция `input()` приостанавливает выполнение
- Запрос на фронтенд
- Поле ввода для пользователя

Control (ROUTER/DEALER)

Управление высокого приоритета

- Команды `shutdown`, `interrupt`
- Работает в отдельном потоке
- Кнопка "Stop" использует этот канал

Heartbeat

REQ/REP (пинг-понг)

Механизм работы:

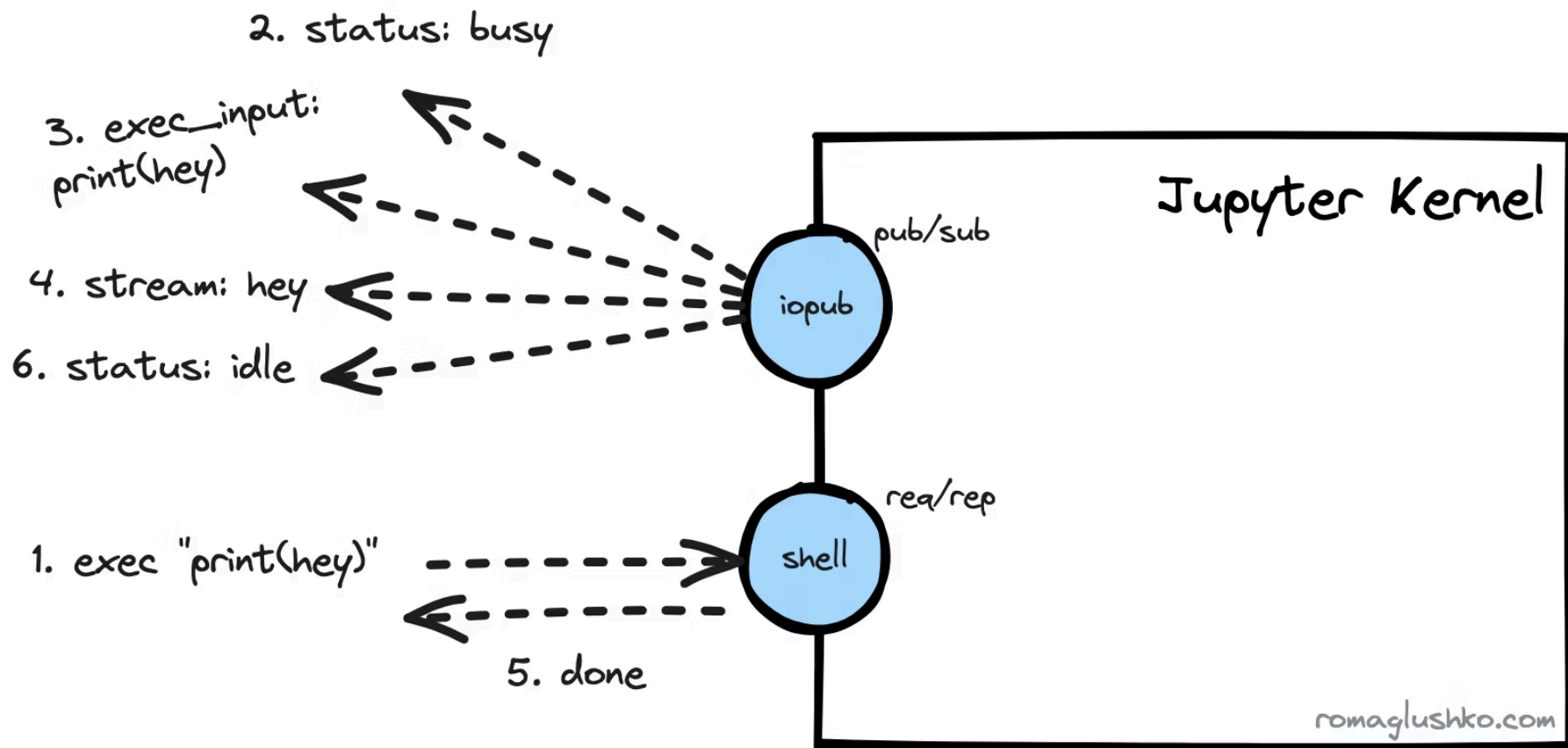
1. Фронтенд отправляет пинг
2. Ядро отвечает понгом
3. Если ответ не приходит → «Dead Kernel»
4. Предложение перезапуска ячейки



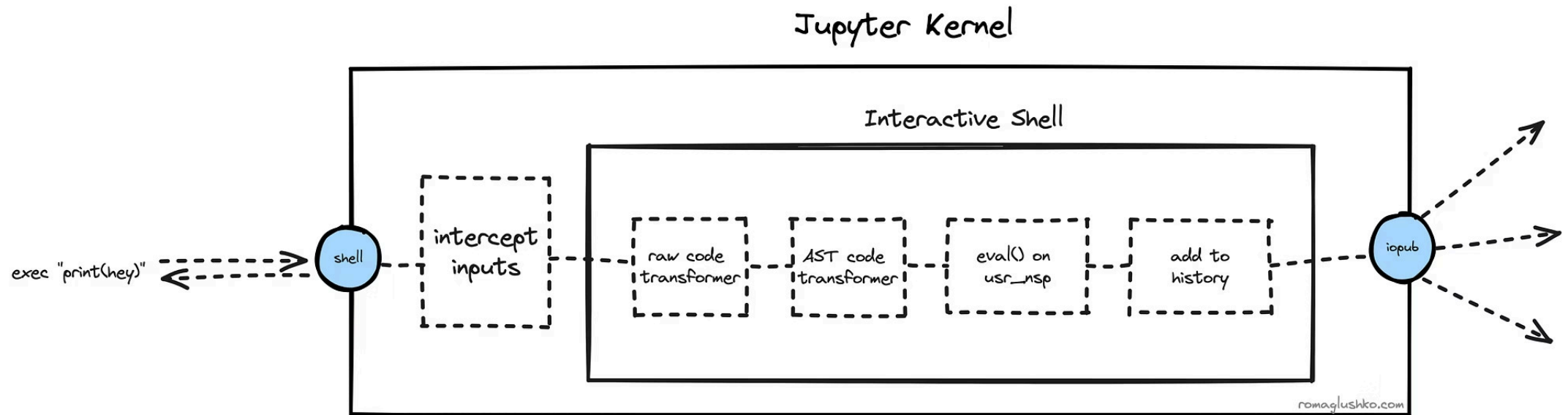
Поток выполнения: шаг за шагом

Что происходит при нажатии Shift+Enter?

Выполнение ячейки



Выполнение ячейки



Выполнение ячейки

01

Формирование

Браузер создает JSON
`execute_request`

02

Передача

`WebSocket` → `Server` → `ZeroMQ`
→ `Shell`

03

Обработка

Ядро переходит в статус `busy`

04

Исполнение

Python интерпретатор выполняет
код

05

Публикация

Вывод отправляется в `IOPub`

06

Рендеринг

Фронтенд отображает результат

07

Завершение

Статус `idle`, индикатор гаснет

ЧАСТЬ II

Инструментарий IPython

Инструменты, выходящие за рамки синтаксиса "ванильного" Python

Категории инструментов IPython



Интроспекция

Доступ к документации и исходному коду



Магические команды

Специальные директивы для управления средой



История

Структурированный доступ к прошлым командам



Отладка

Интерактивный анализ ошибок

Интроспекция объектов

Runtime introspection

Изучение объектов во время выполнения без переключения контекста

?

Docstring функции
или объекта

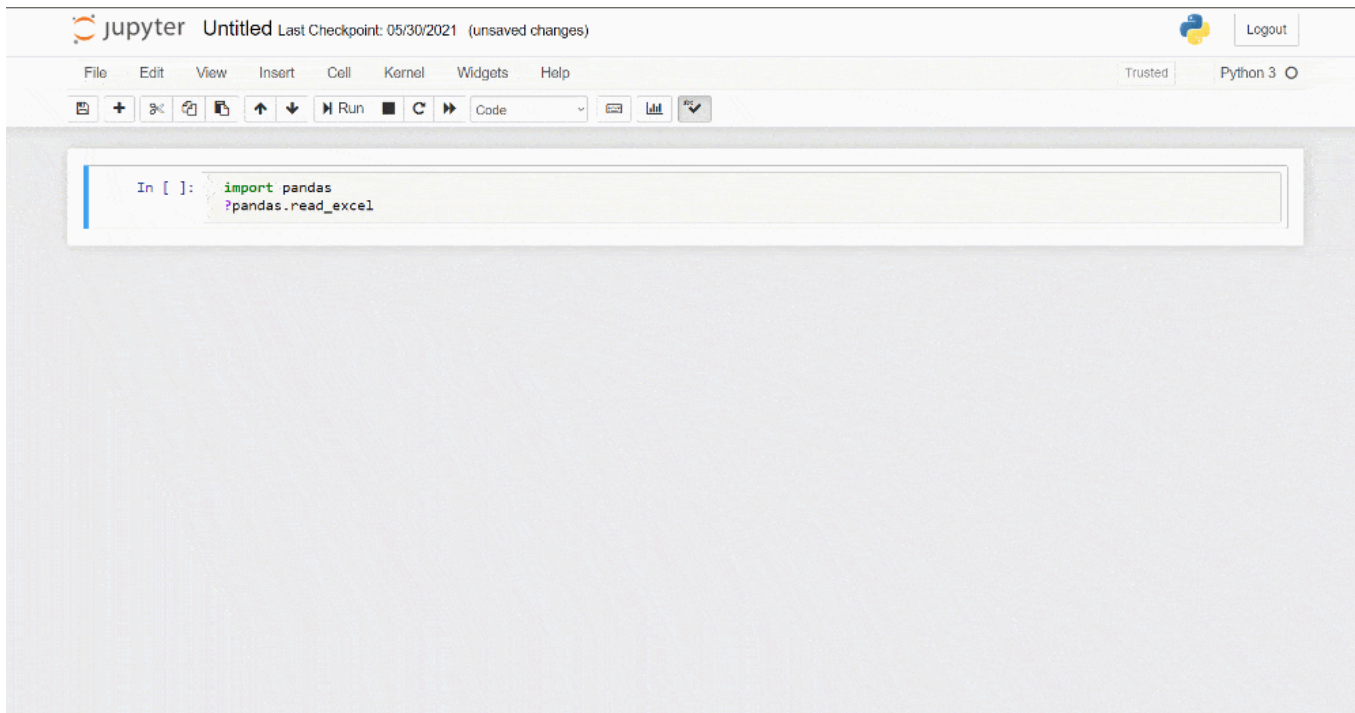
??

Исходный код (если
доступен)

поиск?

Поиск по шаблону в пространстве имен

Одинарный знак вопроса (?)



The screenshot shows a Jupyter Notebook window titled "Untitled" with a last checkpoint of "05/30/2021" and "unsaved changes". The interface includes a top menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and saving. The code cell contains the following Python code:

```
In [ ]: import pandas
        ?pandas.read_excel
```

Доступ к docstring

Результат:

- Сигнатура функции
- Описание параметров
- Примеры использования

Двойной знак вопроса (??)



The screenshot shows a Jupyter Notebook cell with the input `??np.array`. Below the input are two buttons: `+ Code` and `+ Markdown`. The output area displays the docstring for `np.array`, including the function signature, a brief description, and a list of parameters.

```
??np.array
```

+ Code + Markdown

Docstring:
`array(object, dtype=None, copy=True, order='K', subok=False, ndmin=0)`
Create an array.

Parameters

`object` : `array_like`
 An array, any object exposing the array interface, an object whose `__array__` method returns an array, or any (nested) sequence.
`dtype` : `data-type`, optional

Показывает тело функции:

- Полный Python-код
- Метаинформация, применимая к объекту

 **Ограничение:** Не работает для скомпилированных C-расширений (NumPy, Pandas)

Поиск по шаблону с символом *

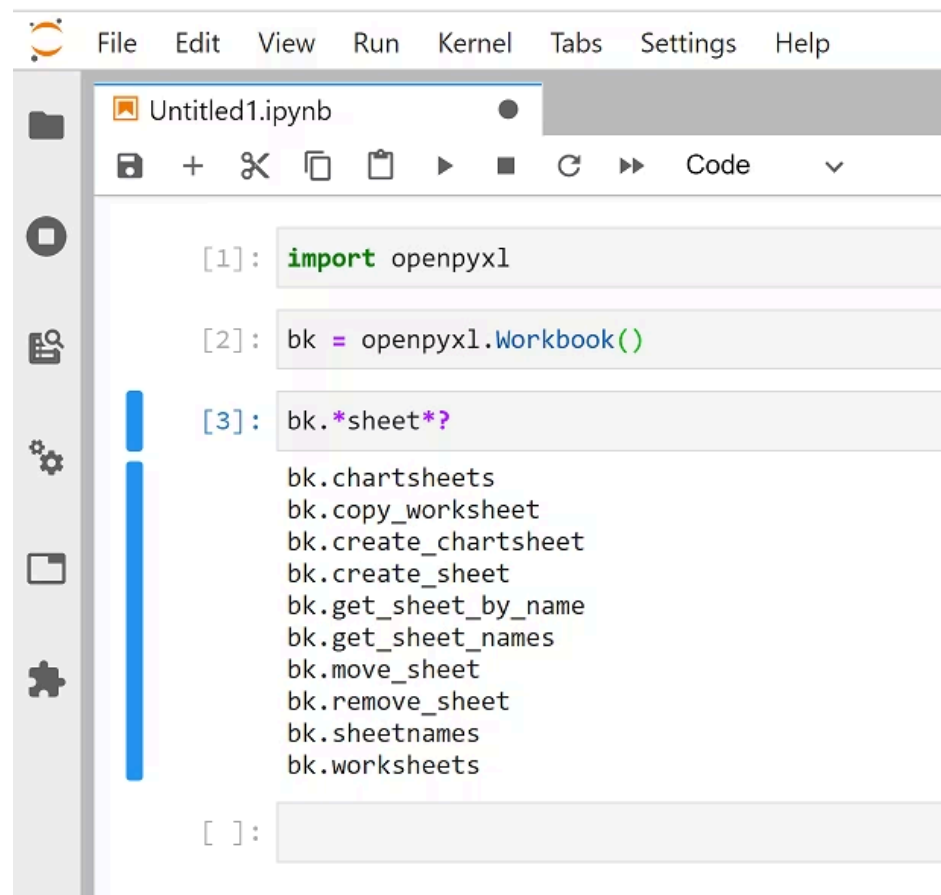
Поиск объектов в namespace

Пример:

```
np.*Array*?
```

Результат:

Список всех атрибутов NumPy, содержащих «Array»



The screenshot shows a Jupyter Notebook window titled 'Untitled1.ipynb'. The interface includes a menu bar (File, Edit, View, Run, Kernel, Tabs, Settings, Help) and a toolbar with icons for file operations and execution. The notebook contains three code cells:

```
[1]: import openpyxl
```

```
[2]: bk = openpyxl.Workbook()
```

```
[3]: bk.*sheet*?
```

The output of the third cell is a list of attributes from the `bk` object that contain the word 'sheet' in their name:

```
bk.chartsheets  
bk.copy_worksheet  
bk.create_chartsheet  
bk.create_sheet  
bk.get_sheet_by_name  
bk.get_sheet_names  
bk.move_sheet  
bk.remove_sheet  
bk.sheetnames  
bk.worksheets
```

Below the output, there is an empty code cell with the prompt `[ ]:`.

MAGIC COMMANDS

Магические команды IPython

Специальные директивы, не являющиеся частью синтаксиса Python

% (строчные)

Применяются к одной строке

%% (ячеечные)

Применяются ко всей ячейке

Пример :

`%time`

Одиночное выполнение – для длительных операций или операций, меняющих состояние

`%prun`

Профилировщик функций – показывает время в каждой функции (`tottime`, `cumtime`)

`%lprun`

Line profiler – детализация до каждой строки кода, требует расширения

%debug – пост-мортем отладка

Запуск после исключения →
попадание внутрь стека в
момент ошибки

Возможности:

- Проверка значений переменных
- Навигация по стеку (up, down)
- Понимание причины без print()

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
<ipython-input-12-4516a42f27b4> in <module>  
      1 x = 1 + 1  
      2 y = 1 - 1  
----> 3 z = x/y  
  
ZeroDivisionError: division by zero  
  
In [*]: %debug  
  
> <ipython-input-12-4516a42f27b4>(3)<module>()  
      1 x = 1 + 1  
      2 y = 1 - 1  
----> 3 z = x/y  
  
ipdb> x  
2  
  
ipdb>   
  
In [ ]: 
```

%xmode – уровни детализации ошибок



Plain

Компактный вывод без
контекста



Context

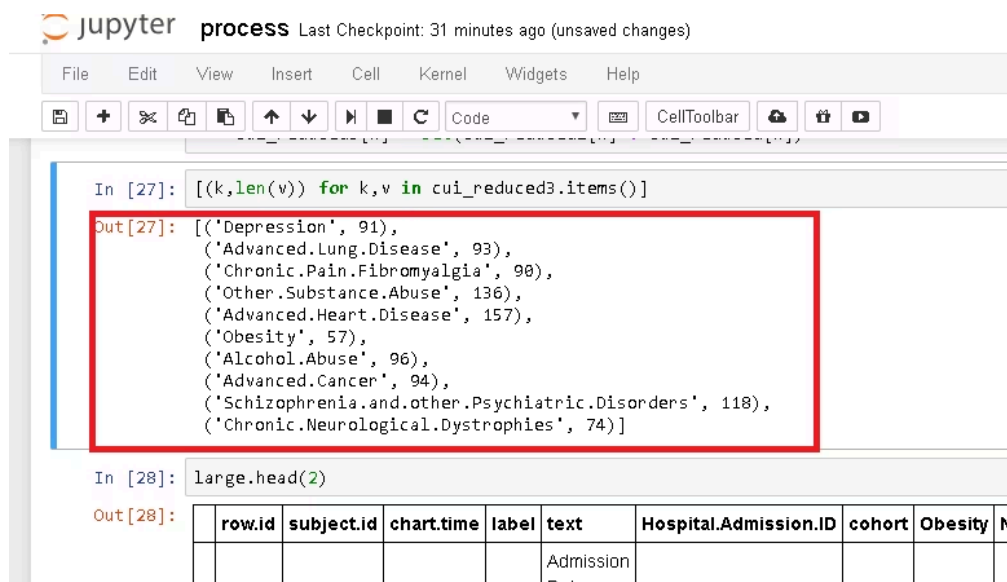
Контекст кода (по
умолчанию)



Verbose

Значения аргументов в стеке
– максимальная детализация

История и объекты In/Out



The screenshot shows a Jupyter Notebook window titled "process" with a status bar indicating "Last Checkpoint: 31 minutes ago (unsaved changes)". The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, code execution, and cell management. The notebook contains two code cells. The first cell, labeled "In [27]:", contains a list comprehension: `[(k, len(v)) for k, v in cui_reduced3.items()]`. The output of this cell, labeled "Out [27]:", is a list of tuples representing disease categories and their counts, enclosed in a red rectangular box. The second cell, labeled "In [28]:", contains the command `large.head(2)`. The output of this cell, labeled "Out [28]:", is a table with 9 columns: row.id, subject.id, chart.time, label, text, Hospital.Admission.ID, cohort, Obesity, and N.

```
In [27]: [(k, len(v)) for k, v in cui_reduced3.items()]
```

```
Out [27]: [('Depression', 91),  
          ('Advanced.Lung.Disease', 93),  
          ('Chronic.Pain.Fibromyalgia', 90),  
          ('Other.Substance.Abuse', 136),  
          ('Advanced.Heart.Disease', 157),  
          ('Obesity', 57),  
          ('Alcohol.Abuse', 96),  
          ('Advanced.Cancer', 94),  
          ('Schizophrenia.and.other.Psychiatric.Disorders', 118),  
          ('Chronic.Neurological.Dystrophies', 74)]
```

```
In [28]: large.head(2)
```

```
Out [28]:
```

row.id	subject.id	chart.time	label	text	Hospital.Admission.ID	cohort	Obesity	N
				Admission				
				Date				

In – список команд

In[27] возвращает код десятой ячейки

Out – словарь результатов

Out[27] возвращает результат десятой ячейки

Позволяет использовать результаты прошлых вычислений без сохранения в переменную

Интеграция с системной оболочкой

```
[1]: ! pwd
```

```
/home/aissak
```

```
[2]: ! ls
```

```
'=0.0.7' datasets Gcode.ipynb my-g  
code ondemand Untitled.ipynb
```

```
[3]: ! cd datasets
```

```
[4]: ! ls
```

```
'=0.0.7' datasets Gcode.ipynb my-g  
code ondemand Untitled.ipynb
```

```
[5]: ! pwd
```

```
/home/aissak
```

Восклицательный знак (!)

Выполнение системных команд:

```
!ls
```

```
!dir
```

Обмен данными:

```
!echo {my_var}
```

```
files = !ls
```

Объект SList с методами .grep()

ЧАСТЬ III

Вычислительный уровень – NumPy

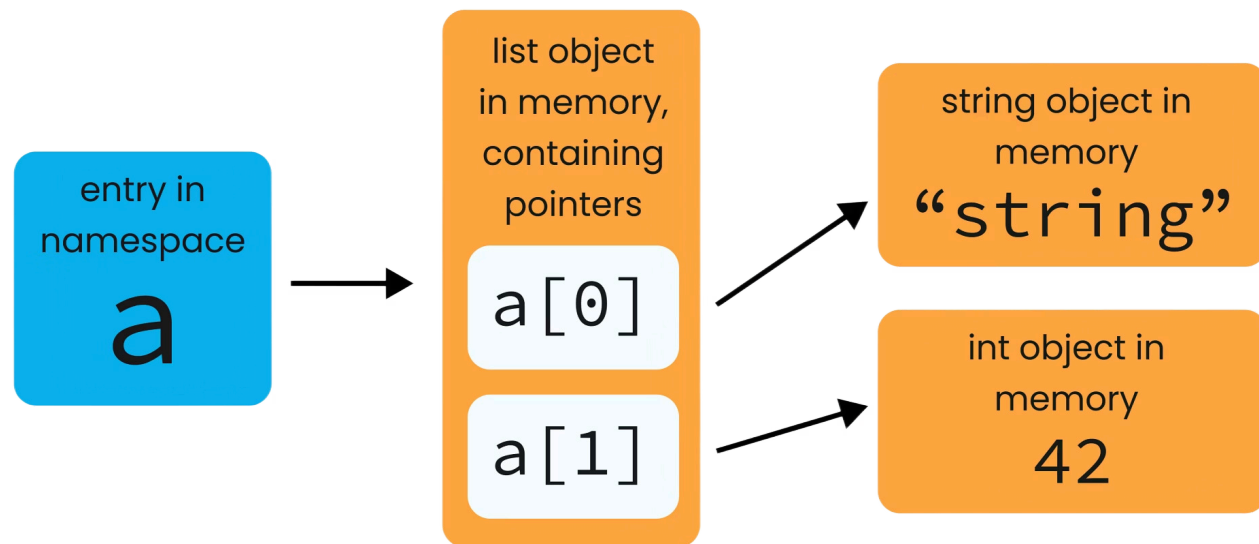
Производительность и устройство памяти

Списки Python vs массивы NumPy

Почему Python медленный, а NumPy быстрый?

Ответ: организация памяти и типизация

Проблема списков Python



Гетерогенность =
накладные расходы

Список может содержать:

```
[42, "string", 3.14]
```

Хранение:

- Указатели на объекты (не сами данные)
- Каждый элемент – PyObject
- 1 байт данных → 28 байт памяти
- Промахи кэша процессора

Структура PyObject

1

Счетчик ссылок
Для сборщика мусора

2

Код типа
Указание, что это int/str/float

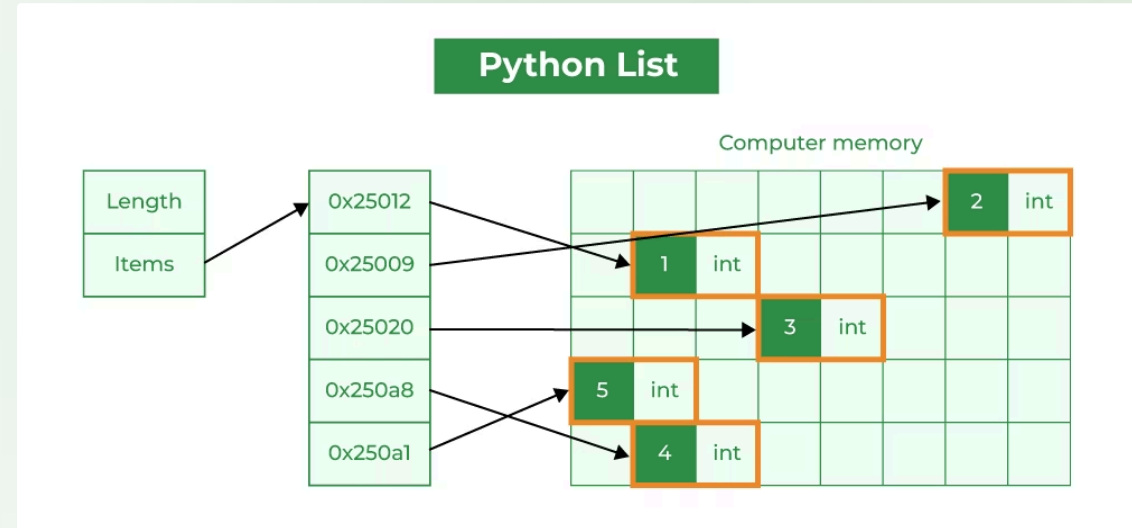
3

Размер
Метаданные объекта

4

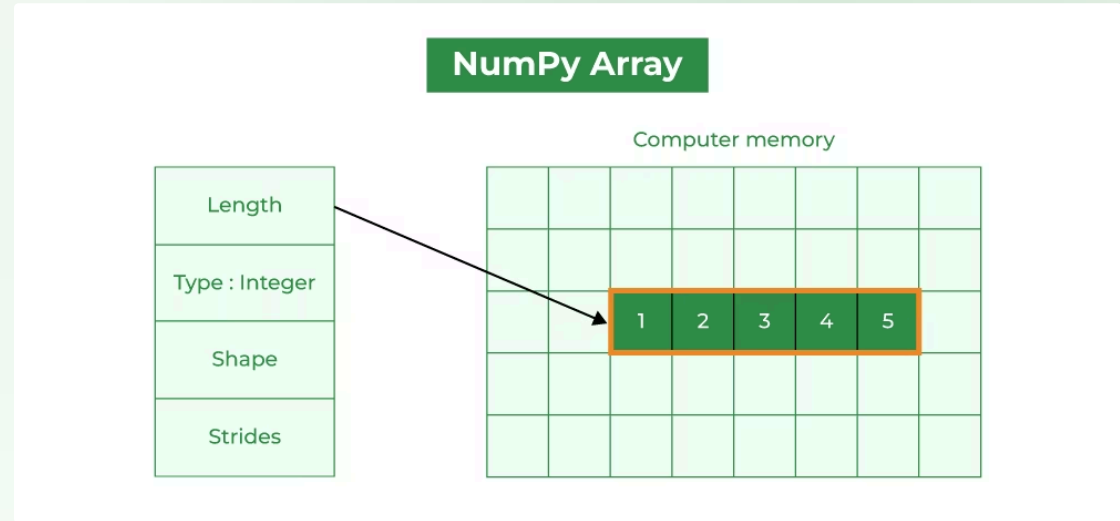
Значение
Собственно данные

Объекты разбросаны по куче – процессор «прыгает» по памяти



NumPy : непрерывная память

Гомогенность увеличивает производительность



Архитектура ndarray

Ограничение → преимущество

Все элементы одного типа

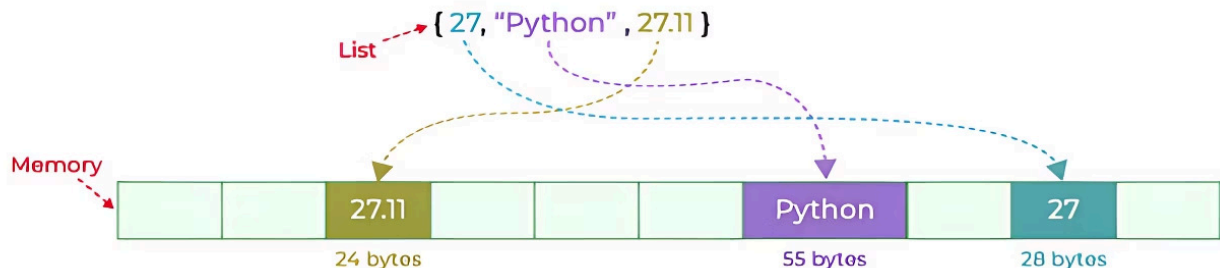
- `int32`, `float64`, и т.д.
- Непрерывный блок памяти
- Как в C или Fortran

Эффективность по памяти

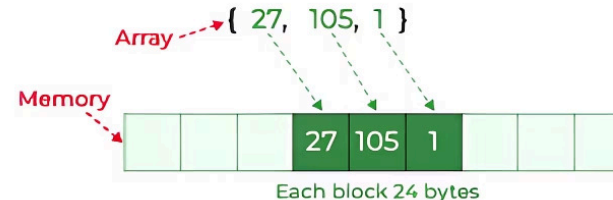
1 млн `int32` = 4 МБ в NumPy

1 млн `int` в списке Python = 28+ МБ

Экономия в 7 раз



Python List



NumPy Array

- Memory size in bytes are in Mac OS X.

Чтобы ваши циклы выполнялись так же быстро как в молодости нужен простой советский...



Unboxed Data

Данные хранятся «как есть» (raw bytes), без накладных расходов объектов Python



Locality of Reference

Последовательное
расположение → эффективная
загрузка в кэш процессора



Векторизация (SIMD)

Одна инструкция процессора
обрабатывает 4-8 чисел
одновременно

Выводы и что можно почитать

